МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО

КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ

З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ

«МЕТОДИ ТА СИСТЕМИ ОБЧИСЛЮВАЛЬНОГО ІНТЕЛЕКТУ»

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірив: старший викладач кафедри ОМНІ Юдіна А. Л.

Кременчук 2026

ЛАБОРАТОРНА РОБОТА № 3

Тема: Одношаровий персептрон. Виконання логічних операцій

Мета: отримати навички алгоритмізації та програмної реалізації персептрона, що розпізнає бінарні образи.

Порядок виконання роботи:

У роботі необхідно описати алгоритм і побудувати програму навчання і тестування персептрона, що обчислює логічні функції. Варіанти завдань наведені у табл. 3.2. Потрібно розгорнути обрану за варіантом формулу та заповнити відповідну таблицю істинності, яка буде слугувати джерелом для генерування прикладів.

Таблиця 3.1 – Варіанти завдання

№ вар.	Функція для обчислення	Функція активації
15	$x1 \cdot x2 \cdot \bar{x}3 + x1 \cdot \bar{x}2 \cdot x3$	3.3

Хід роботи:

1. Таблиця істинності:

Таблиця 3.2 – Таблиця істинності

x1	**x2**	**x3**	**Y**
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

2. Схема персептрона:

w0 – зміщення (bias)

w1 – вага x1

w2 – вага x2

w3 – вага x3

Зважена сума:

$$S = w0 + w1x1 + w1x2 + wx3$$

Функція активації:

$$Y = \frac{1}{1 + e^{-S}}$$

3. Алгоритм навчання

1) Створюємо таблицю істинності функції.

2) Ініціалізуємо вагові коефіцієнти випадковими числами.

W0, W1, W2, W3

3) Для кожного рядка таблиці істинності генеруємо навчальні приклади.

Для цього:

– якщо значення ознаки 0

X = random * 0.49

– якщо 1

X = 0.5 + random * 0.5

4) Обчислюємо зважену суму:

$$S = w0 + w1x1 + w1x2 + wx3$$

5) Пропускаємо суму через функцію активації:

$$Y = \frac{1}{1 + e^{-S}}$$

6) Обчислюємо похибку:

$$d = T - Y$$

де T – правильна відповідь з таблиці істинності.

7) Коригуємо ваги (дельта правило):

$$W_j = W_j + \alpha d X_j$$

де α – коефіцієнт навчання

8) Обчислюємо квадратичну похибку

$$D = d^2$$

9) Після проходження всієї вибірки обчислюємо повну похибку

$$E = \frac{\sum D}{N}$$

де N – кількість прикладів.

4. Алгоритм тестування

1) Генеруємо тестові приклади так само, як для навчання.

2) Для кожного прикладу обчислюємо:

$$S = w0 + w1x1 + w1x2 + wx3$$

3) Обчислюємо вихід нейрона

$$Y = \frac{1}{1 + e^{-S}}$$

4) Обчислюємо похибку

$$d = T - Y$$

5) Обчислюємо квадратичну похибку

$$D = d^2$$

6) Після тестування всіх прикладів рахуємо

$$E_{test}$$

7) Порівнюємо

$$E_{train}$$

$$E_{test}$$

Якщо вони близькі, то навчання успішне.

5. Скрипт програми:

```
import random
import math

# ==============================
# 1. Таблиця істинності функції
# x1 x2 x3 Y
# ==============================

M = [
    [0,0,0,0],
    [0,0,1,0],
    [0,1,0,0],
    [0,1,1,0],
    [1,0,0,0],
    [1,0,1,1],
    [1,1,0,1],
    [1,1,1,0]
```

```
]

# Кількість стовпців у таблиці
L = 4


# ==============================
# 2. Параметри навчання
# ==============================

alpha = 0.1         # коефіцієнт навчання (швидкість зміни ваг)
epochs = 100        # кількість циклів навчання
examples = 20       # кількість прикладів для кожного рядка таблиці


# ==============================
# 3. Початкова ініціалізація ваг
# ==============================

# Вагові коефіцієнти генеруються випадково
W = [random.uniform(-1,1) for i in range(L)]


# ==============================
# 4. Сигмоїдальна функція активації
# ==============================

# Функція активації перетворює зважену суму
# у значення виходу нейрона
def activation(S):
    return 1/(1+math.exp(-S))


# ==============================
# 5. Генерація вхідних сигналів
# ==============================

# Функція створює "реальні" значення сигналів
# для логічного 0 або 1
def generate_input(value):

    if value == 0:
        # якщо логічний 0 → генеруємо число у межах 0 - 0.49
        return 0.49 * random.random()
    else:
        # якщо логічна 1 → генеруємо число у межах 0.5 - 1
        return 0.5 * random.random() + 0.5


# ==============================
# 6. НАВЧАННЯ ПЕРСЕПТРОНА
# ==============================

print("=== ПОЧАТОК НАВЧАННЯ ===\n")

# цикл навчання
for epoch in range(epochs):

    total_error = 0   # змінна для накопичення повної похибки

    # перебір рядків таблиці істинності
    for row in M:

        # генерація прикладів для кожного рядка
        for k in range(examples):
```

```
            # формуємо вектор вхідних сигналів
            # X0 = 1 використовується для зміщення (bias)
            X = [1]

            for j in range(3):
                X.append(generate_input(row[j]))

            # правильний вихід з таблиці істинності
            T = row[3]

            # ==============================
            # обчислення зваженої суми
            # ==============================

            S = 0
            for j in range(L):
                S += W[j] * X[j]

            # ==============================
            # обчислення виходу нейрона
            # ==============================

            Y = activation(S)

            # ==============================
            # обчислення похибки
            # ==============================

            d = T - Y

            # ==============================
            # корекція ваг (дельта правило)
            # ==============================

            for j in range(L):
                W[j] = W[j] + alpha * d * X[j]

            # додаємо квадрат похибки
            total_error += d**2

    # середня квадратична похибка
    E = total_error / (len(M) * examples)

    print("Цикл навчання:", epoch+1)
    print("Вагові коефіцієнти:", W)
    print("Похибка:", E)

print("=== НАВЧАННЯ ЗАВЕРШЕНО ===\n")


# ==============================
# 7. ТЕСТУВАННЯ ПЕРСЕПТРОНА
# ==============================

print("=== ТЕСТУВАННЯ ===\n")

test_error = 0

# перебір рядків таблиці істинності
for row in M:

    for k in range(examples):

        # формування вхідного вектора
```

```
        X = [1]

        for j in range(3):
            X.append(generate_input(row[j]))

        # правильне значення виходу
        T = row[3]

        # обчислення зваженої суми
        S = 0
        for j in range(L):
            S += W[j] * X[j]

        # обчислення виходу нейрона
        Y = activation(S)

        # обчислення похибки
        d = T - Y

        test_error += d**2

# середня похибка на тестовій вибірці
Etest = test_error / (len(M) * examples)

print("Похибка на тестовій вибірці:", Etest)
```

Результат:

```
=== ПОЧАТОК НАВЧАННЯ ===

Цикл навчання: 1
Вагові коефіцієнти: [-0.8557184755360777, 0.6134396877049353, -0.28294755560539386, -0.6672088807327434]
Похибка: 0.17552015618900407
Цикл навчання: 2
Вагові коефіцієнти: [-0.9467024577802141, 0.9675559271402121, -0.6200946409768353, -0.6036183281264851]
Похибка: 0.13118335530985448
Цикл навчання: 3
Вагові коефіцієнти: [-1.0570551126235752, 1.3011297292910584, -0.9055488156007471, -0.443425189666457]
Похибка: 0.1284495877817181
Цикл навчання: 4
Вагові коефіцієнти: [-1.1496388800531598, 1.4845493857016145, -1.0660291576834324, -0.4590736270142033]
Похибка: 0.1336887616664042
Цикл навчання: 5
Вагові коефіцієнти: [-1.1919861049694018, 1.7696426619509562, -1.3369418851033712, -0.46223579287207545]
Похибка: 0.12226235788873929
Цикл навчання: 6
Вагові коефіцієнти: [-1.2733846196236982, 1.9578062773757703, -1.5159398836367102, -0.4689648163044025]
Похибка: 0.12488575984837406
Цикл навчання: 7
Вагові коефіцієнти: [-1.2074374064464815, 2.2726791482356488, -1.7554744492416923, -0.39510158224865505]
Похибка: 0.12050469478101782
Цикл навчання: 8
Вагові коефіцієнти: [-1.2694774441065229, 2.631676421222154, -1.9442015991678734, -0.29955327172684043]
Похибка: 0.1201772049209507
Цикл навчання: 9
Вагові коефіцієнти: [-1.5433470536610856, 2.768000315025209, -2.086708968027446, -0.2880077497166897]
Похибка: 0.11842327371368373
Цикл навчання: 10
Вагові коефіцієнти: [-1.6454073472807105, 2.762096800048071, -2.1673762529477494, -0.3019676870308406]
Похибка: 0.1376099879695912
```

Рисунок 3.1 – Результат роботи програми

```
Цикл навчання: 95
Вагові коефіцієнти: [-2.8206817013416847, 5.301632295879271, -2.533169630688197, -0.38020297364768096]
Похибка: 0.12388993447788432
Цикл навчання: 96
Вагові коефіцієнти: [-2.933728328047116, 5.080915346392029, -2.492313836497436, -0.2683048135775255]
Похибка: 0.15425287532802615
Цикл навчання: 97
Вагові коефіцієнти: [-2.949515095609439, 5.17294996655193, -2.4670068138295247, -0.2593535119801173]
Похибка: 0.11680520656762687
Цикл навчання: 98
Вагові коефіцієнти: [-2.981840397634461, 5.083153800761533, -2.4498788930996622, -0.13025739196640845]
Похибка: 0.13986069265008425
Цикл навчання: 99
Вагові коефіцієнти: [-2.994148462894256, 5.1410758160006065, -2.482162372745833, -0.13057239953668143]
Похибка: 0.11566625220670965
Цикл навчання: 100
Вагові коефіцієнти: [-3.091945552974188, 5.104885318357034, -2.556095836685609, -0.17714555117879624]
Похибка: 0.12429358644468524
=== НАВЧАННЯ ЗАВЕРШЕНО ===

=== ТЕСТУВАННЯ ===

Похибка на тестовій вибірці: 0.16659377749213403
```

Рисунок 3.2 – Продовження результату роботи програми

Висновок: на цій лабораторній роботі ми отримали навички алгоритмізації та програмної реалізації персептрона, що розпізнає бінарні образи. Склали схему персептрона, навели словесний опис алгоритму навчання по пунктах, склали опис алгоритму тестування по пунктах, склали програму, яка генерує приклади для навчання і тестування персептрона.

Контрольні питання:

1. Принцип роботи нейрона за моделлю Мак–Каллока.

Нейрон за моделлю Мак–Каллока–Піттса є спрощеною математичною моделлю біологічного нейрона. Він отримує кілька вхідних сигналів, кожен з яких множиться на відповідний ваговий коефіцієнт. Після цього обчислюється зважена сума всіх сигналів. Якщо отримане значення перевищує певний поріг, нейрон видає сигнал на виході, інакше вихід дорівнює нулю. Таким чином нейрон виконує просту операцію прийняття рішення на основі вхідних даних.

2. Етапи алгоритму навчання одношарового персептрона.

Алгоритм навчання одношарового персептрона складається з кількох етапів. Спочатку задаються початкові значення вагових коефіцієнтів, зазвичай випадковим

чином. Потім на вхід нейрона подаються навчальні приклади. Для кожного прикладу обчислюється зважена сума сигналів та визначається вихід нейрона за допомогою функції активації. Далі обчислюється похибка як різниця між правильним значенням і отриманим результатом. На основі цієї похибки коригуються вагові коефіцієнти. Процес повторюється багато разів, поки похибка не стане достатньо малою.

3. Принцип створення прикладів для навчання в задачі обчислення логічних функцій.

У задачах обчислення логічних функцій приклади для навчання формуються на основі таблиці істинності. Кожен рядок таблиці містить значення вхідних змінних та відповідний правильний результат функції. Для моделювання реальних сигналів логічні значення 0 та 1 можуть замінюватися випадковими числами з певних інтервалів. Наприклад, для логічного нуля генеруються значення в межах від 0 до 0.49, а для логічної одиниці – від 0.5 до 1. Це дозволяє навчити нейрон працювати з неперервними сигналами.

4. Алгоритм тестування персептрона на тестовій вибірці.

Після завершення навчання виконується тестування персептрона. Для цього використовується тестова вибірка, яка містить приклади з відомими правильними результатами. Для кожного прикладу обчислюється зважена сума вхідних сигналів та визначається вихід нейрона. Отриманий результат порівнюється з правильним значенням, після чого обчислюється похибка. Підсумкова похибка на тестовій вибірці показує, наскільки добре навчений персептрон може узагальнювати знання та працювати з новими даними.